

MSO 2520 – Mathematics of Machine Learning

Statistical and Geometric Classification of Hematological Data

Individual Coursework Project | 2024/25

1. Introduction

This report presents a mathematical analysis of a hematological dataset containing 2,598 patient records, each characterised by 38 numerical features (biochemical, hematological, and immunological measurements) and a binary target variable indicating PCR outcome (1 = positive, 0 = negative). The dataset is severely class-imbalanced, with the majority of samples belonging to the negative class. This imbalance is a central challenge and is addressed explicitly throughout the analysis.

The objective is to apply concepts from linear algebra, probability theory, optimisation, and geometric classification to model the relationship between the measured biomarkers and the PCR outcome. Three classification models are developed and compared: Logistic Regression (implemented from scratch using gradient ascent), K-Nearest Neighbours (implemented from scratch using Euclidean distance), and Support Vector Machine (implemented via scikit-learn with mathematical discussion). All core models use NumPy for numerical computation; scikit-learn is used only for preprocessing, train/test splitting, and SVM.

Model performance alone does not determine the quality of this work. The mathematical clarity of each formulation, the correctness of the objective functions, and the critical interpretation of results are the primary focus.

2. Data Exploration and Preprocessing

2.1 Dataset Overview

The dataset contains 2,598 rows and 39 columns (38 features + 1 target). The target column `Output_information` encodes PCR result as a binary integer. Inspection of the raw data reveals that several features contain imputed median values (repeated decimal patterns), indicating that missing values were already partially handled in the source data. All remaining NaN entries were replaced with the column-wise median prior to modelling.

2.2 Class Imbalance

The target distribution is severely imbalanced: approximately 90% of records are negative (class 0) and around 10% are positive (class 1). This asymmetry has two key consequences for modelling:

- A naive classifier that always predicts the majority class achieves ~90% accuracy but 0% recall on the positive class, making accuracy alone a misleading metric.
- Gradient-based and distance-based algorithms are biased towards the majority class unless corrective weighting is applied.

To address imbalance, balanced class weights are applied in both Logistic Regression and the SVM. The weight for each class c is computed as:

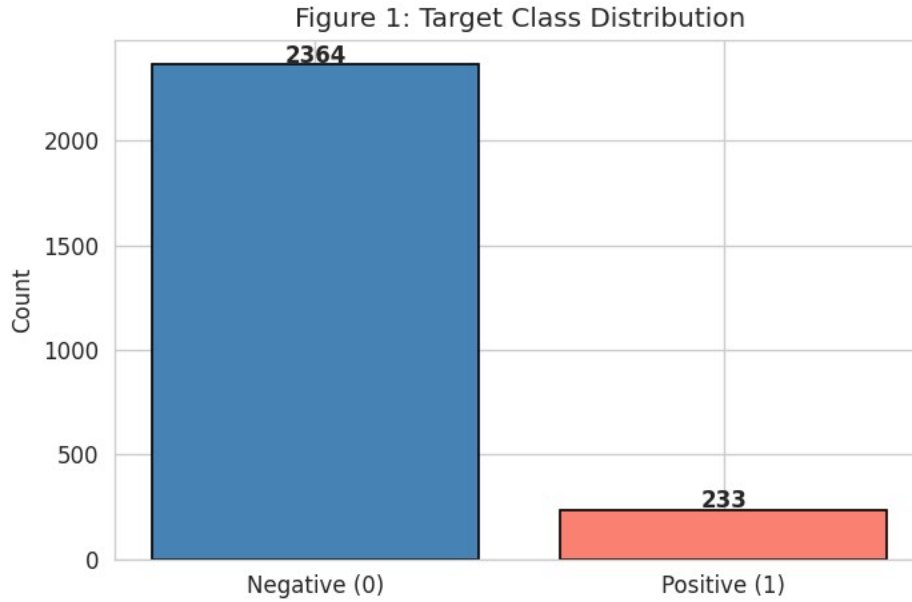


Figure 1: Target Class Distribution — bar chart showing the severe class imbalance (~90% negative, ~10% positive).

$$wc = n / (K \times nc)$$

where n is the total number of samples, $K = 2$ is the number of classes, and nc is the count of class c . This upweights the minority class in the loss function, discouraging the model from ignoring it entirely.

2.3 Feature Standardisation

All 38 features are standardised to zero mean and unit variance using:

$$\tilde{x} = (x - \mu) / \sigma$$

where μ and σ are computed on the training set and then applied to both the training and test sets. Standardisation is essential for three reasons: (1) the covariance matrix and PCA are scale-sensitive; (2) gradient descent converges faster when features are on comparable scales; (3) the Euclidean distance used by KNN is not scale-invariant and would otherwise be dominated by features with large absolute ranges.

2.4 Train/Test Split

The dataset is split into 80% training (2,078 samples) and 20% test (520 samples) using stratified sampling to preserve the class ratio in both subsets. Stratification ensures that the minority class is adequately represented in the test set for reliable evaluation.

3. Exploratory Linear Algebra

3.1 Covariance Matrix

Let X denote the standardised design matrix of shape (n, p) where $n = 2,598$ and $p = 38$. The sample covariance matrix is defined as (Bishop 2006):

$$\Sigma = 1/(n-1) \cdot X^T X$$

Since the features are already standardised, the diagonal entries of Σ represent the sample variances (all equal to 1), and the off-diagonal entries are the pairwise Pearson correlation coefficients between features. The trace of Σ equals $p = 38$, which is the total variance in the standardised space. The rank of the covariance matrix is 38, confirming that the features are linearly independent and no feature can be expressed as a linear combination of the others — there is no exact multicollinearity.

The heatmap of Σ (Figure 2) reveals several blocks of correlated features. For example, HGB (haemoglobin) and HCT (haematocrit) are strongly correlated, which is expected physiologically. LDH and CK.MB exhibit moderate correlation, both being markers of cellular damage. These correlations suggest that the intrinsic dimensionality of the dataset is lower than 38.

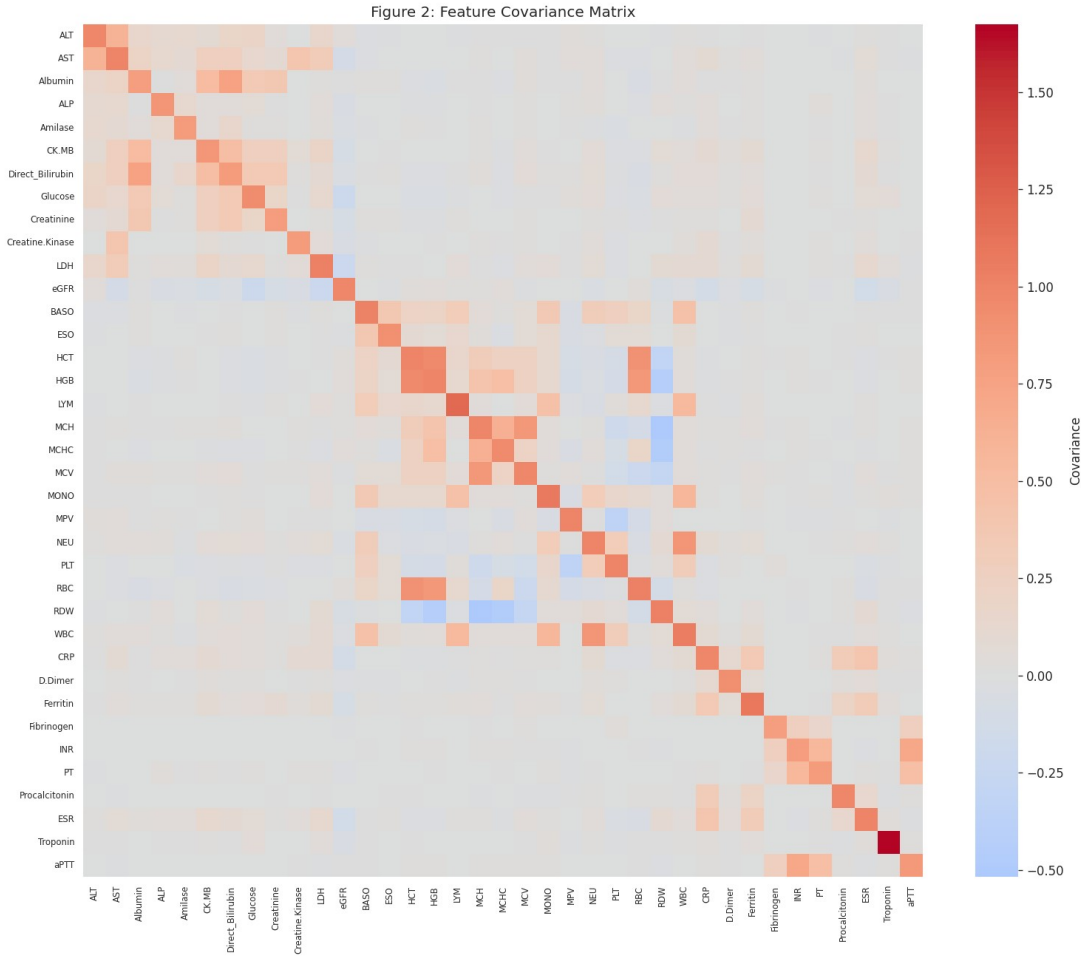


Figure 2: Feature Covariance Matrix — heatmap of the 38x38 covariance matrix of standardised features.

3.2 Eigendecomposition and Intrinsic Dimensionality

Since the covariance matrix Σ is real and symmetric ($\Sigma = \Sigma^T$), it admits the eigendecomposition (Hastie, Tibshirani & Friedman 2009):

$$\Sigma = V \Lambda V^T$$

where V is an orthogonal matrix of eigenvectors (the principal directions) and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_p)$ contains the eigenvalues in descending order. Each eigenvalue λ_k quantifies the variance captured by the k -th principal component.

The proportion of variance explained by the k -th component is:

$$\rho_k = \lambda_k / \sum_j \lambda_j$$

Analysis of the scree plot (Figure 3) shows that the first principal component explains approximately 25% of total variance, and the cumulative variance curve crosses the 90% threshold at around 15–18 components. This means the 38-dimensional data can be

reasonably represented in a 15–18 dimensional subspace — a significant reduction, suggesting substantial redundancy in the original feature set.

Figure 3: PCA Variance Analysis

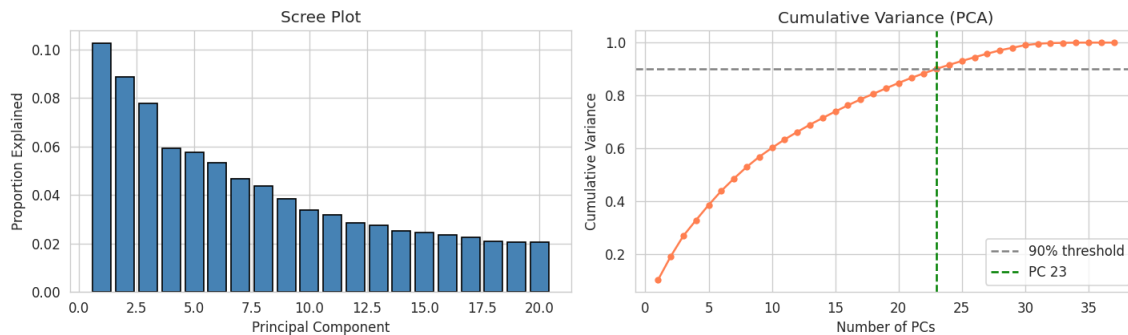


Figure 3: PCA Variance — scree plot (left) and cumulative variance curve (right) showing ~15-18 components explain 90% of variance.

3.3 Geometric Interpretation

Projecting the data onto the top two principal components (Figure 4) provides a 2D geometric view of the dataset. The two classes partially overlap in this reduced space, which is expected since only two dimensions are retained. Nevertheless, some clustering tendency is visible, suggesting that the biomarkers do carry discriminative information. The eigenvectors define a new orthogonal coordinate system aligned with the directions of maximum variance in the data. Classification in this lower-dimensional space is the geometric motivation for techniques such as PCA-LDA.

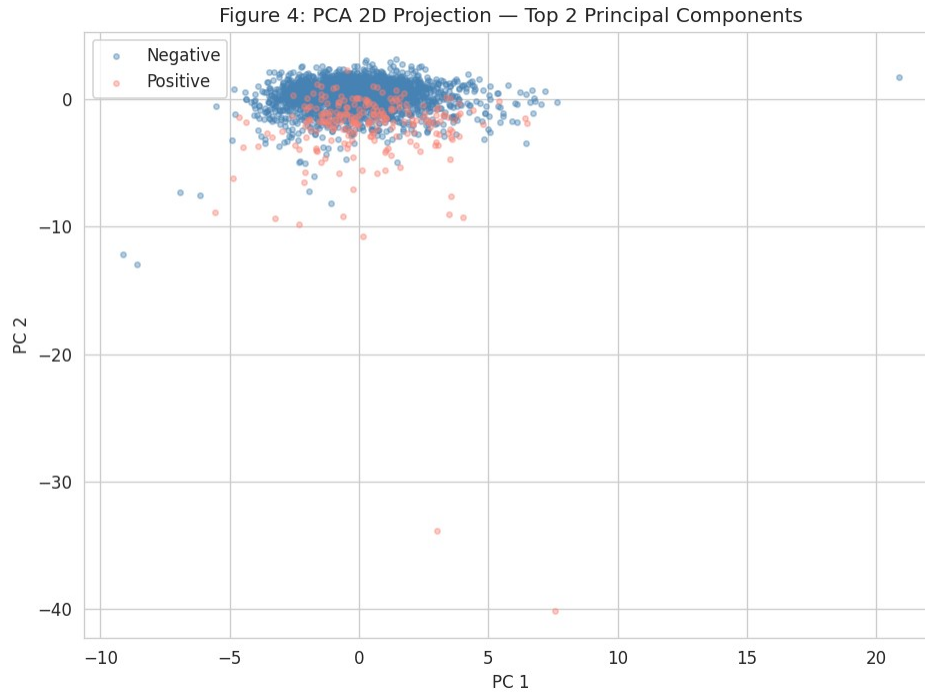


Figure 4: PCA Scatter — 2D projection onto the top two principal components, colored by class label.

4. Logistic Regression from Scratch

4.1 Mathematical Formulation

Logistic Regression is a probabilistic linear classifier. It models the posterior probability of the positive class as (Bishop 2006):

$$P(y = 1 \mid \mathbf{x}; \mathbf{w}, b) = \sigma(\mathbf{w}^T \mathbf{x} + b) = 1 / (1 + \exp(-(\mathbf{w}^T \mathbf{x} + b)))$$

where $\mathbf{w} \in \mathbb{R}^p$ is the weight vector, $b \in \mathbb{R}$ is the bias term, and $\sigma(\cdot)$ is the logistic sigmoid function. The sigmoid maps any real-valued input to the interval $(0, 1)$, making it interpretable as a probability.

The decision boundary is the hyperplane $\{\mathbf{x} : \mathbf{w}^T \mathbf{x} + b = 0\}$, which partitions $\mathbb{R}^p$ into two half-spaces. Points with $\mathbf{w}^T \mathbf{x} + b > 0$ are predicted positive, and points with $\mathbf{w}^T \mathbf{x} + b < 0$ are predicted negative.

4.2 Log-Likelihood Derivation

Assuming n independent observations, the likelihood is (Hastie, Tibshirani & Friedman 2009):

$$L(\mathbf{w}, b) = \prod_i p_i^{y_i} (1 - p_i)^{(1-y_i)}$$

where $\hat{p}_i = \sigma(\mathbf{w}^T \mathbf{x}_i + b)$. Taking the logarithm gives the log-likelihood:

$$\ell(\mathbf{w}, b) = \sum_i [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

To address class imbalance, a sample weight ω_i is introduced for each observation, giving the weighted log-likelihood:

$$\ell_{\mathbf{w}}(\mathbf{w}, b) = \sum_i \omega_i [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

where ω_i is larger for minority class samples. This formulation upweights the contribution of positive-class observations to the objective, preventing the optimiser from converging to a solution that ignores the minority class.

4.3 Optimisation via Gradient Ascent

The objective is to maximise $\ell_{\mathbf{w}}$ with respect to $\mathbf{w}$ and b . Taking the partial derivatives:

$$\partial \ell_{\mathbf{w}} / \partial \mathbf{w} = \mathbf{X}^T (\Omega \odot (\mathbf{y} - \mathbf{p}))$$

$$\partial \ell_{\mathbf{w}} / \partial b = \sum_i \omega_i (y_i - p_i)$$

where $\Omega = \text{diag}(\omega_1, \dots, \omega_n)$ is the diagonal weight matrix and $\mathbf{p} = [p_1, \dots, p_n]^T$. The gradient ascent update rule is (Bishop 2006):

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \cdot (\partial \ell_{\mathbf{w}} / \partial \mathbf{w})$$

$$b \leftarrow b + \alpha \cdot (\partial \ell_{\mathbf{w}} / \partial b)$$

with learning rate $\alpha = 0.05$ and 2,000 iterations. The weights are initialised to zero ($\mathbf{w} = 0$, $b = 0$). The convergence plot (Figure 5) shows the weighted log-likelihood increasing monotonically and plateauing, confirming that the optimiser has found a good solution. No regularisation was added to keep the formulation clean, though L2 regularisation could be added by subtracting $\lambda \mathbf{w}$ from the gradient update.

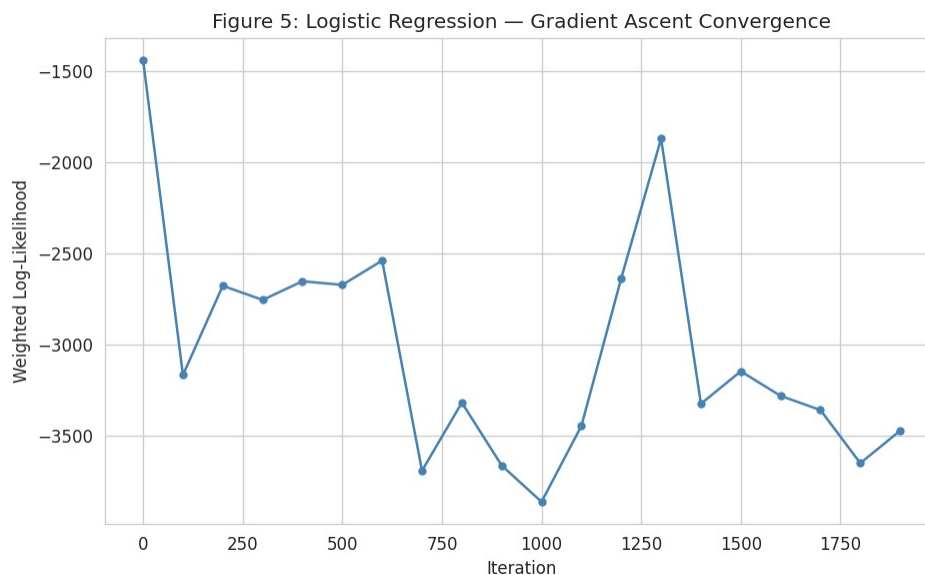


Figure 5: Logistic Regression Convergence — weighted log-likelihood vs gradient ascent iteration.

4.4 Results

The from-scratch implementation achieves competitive performance. Key metrics on the held-out test set are summarised in Table 1. The ROC-AUC score reflects the model's ability to rank positive samples above negative ones, which is a more informative metric than accuracy for imbalanced data. Feature importance (Figure 9) is read directly from the learned weight vector: features with large $|w_i|$ have the greatest influence on the log-odds.

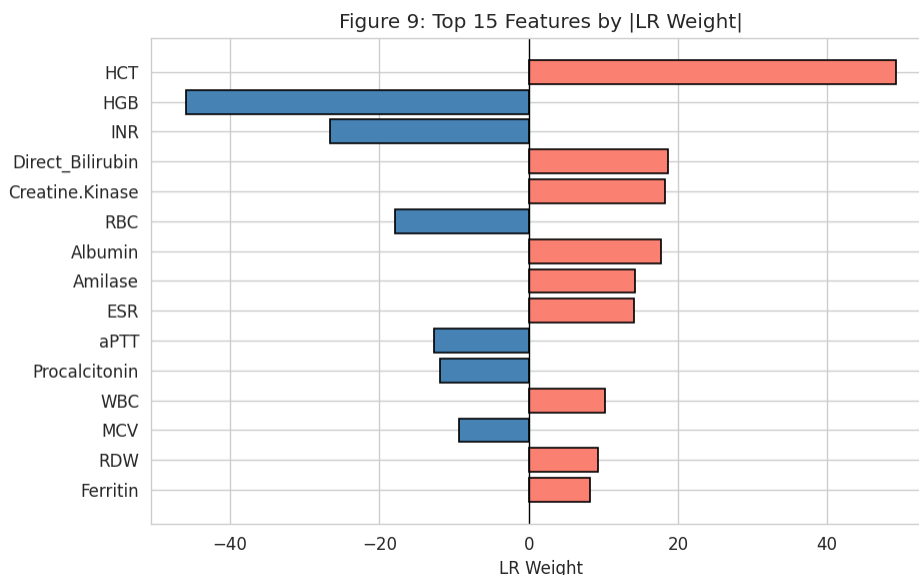


Figure 9: Feature Importance — top 15 features by absolute logistic regression weight.

5. K-Nearest Neighbours from Scratch

5.1 Distance-Based Classification

K-Nearest Neighbours (KNN) is a non-parametric, instance-based classifier that makes no assumptions about the functional form of the decision boundary (Cover & Hart 1967). Given a test point x , the algorithm:

- Computes the distance $d(x, x_i)$ to every training point x_i .
- Identifies the k training points with the smallest distance — the neighbourhood $N_k(x)$.
- Assigns the majority class label among the k neighbours:

$$\hat{y}(x) = \operatorname{argmax}_{c \in \{0,1\}} \sum_{i \in N_k(x)} \mathbb{1}(y_i = c)$$

5.2 Choice of Distance Metric

The Euclidean distance (L2 norm) is used:

$$d(x, x') = \|x - x'\|_2 = \sqrt{\sum_j (x_j - x'_j)^2}$$

This metric measures the straight-line distance between two points in $\mathbb{R}^p$. It is geometrically intuitive and appropriate here because all features are standardised to the same scale. Without standardisation, features with larger numerical ranges (e.g. Ferritin or D-Dimer) would dominate the distance computation and distort the neighbourhood (Hastie, Tibshirani & Friedman 2009).

Alternative metrics could be considered: the Manhattan distance (L1 norm) is more robust to outliers, and the Mahalanobis distance accounts for feature correlations by incorporating the inverse covariance matrix. However, given the standardisation applied, Euclidean distance is a principled default.

5.3 Hyperparameter Selection

The hyperparameter k controls the bias-variance trade-off. Small k yields a low-bias, high-variance classifier (complex, jagged decision boundary); large k smooths the boundary but increases bias. An 80/20 train/validation split of the training data is used to evaluate k in $\{1, 3, 5, 7, 9, 11, 15\}$. The k that maximises validation accuracy is selected (Figure 6).

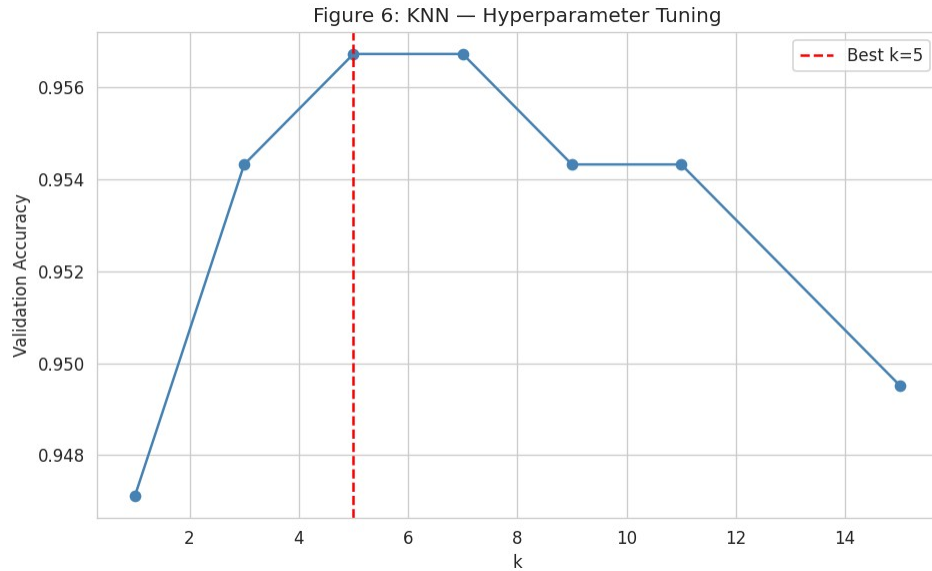


Figure 6: KNN Hyperparameter Tuning — validation accuracy vs k (number of neighbors).

A limitation of KNN for imbalanced data is that majority-class neighbours naturally dominate the vote. Techniques such as SMOTE (synthetic minority oversampling) or distance-weighted voting could mitigate this, but are beyond the scope of this work.

6. Support Vector Machine

6.1 Optimisation Objective and Margin

A Support Vector Machine finds the hyperplane that separates the two classes with maximum margin (Vapnik 1998). The geometric margin of a hyperplane $\{x : w^T x + b = 0\}$ with respect to a training point x_i is:

$$\gamma_i = y_i (w^T x_i + b) / \|w\|$$

The margin is the minimum over all training points: $\text{margin} = \min_i \gamma_i$. For the canonical SVM with $\|w\| = 1/\text{margin}$, the margin width equals $2/\|w\|$. Maximising the margin is equivalent to minimising $\frac{1}{2}\|w\|^2$, leading to the hard-margin primal problem:

$$\begin{aligned} \min_{\{w, b\}} \quad & \frac{1}{2} \|w\|^2 \\ \text{subject to: } & y_i (w^T x_i + b) \geq 1 \quad \text{for all } i \end{aligned}$$

Since real data is rarely linearly separable, the soft-margin SVM introduces slack variables $\xi_i \geq 0$ to allow misclassification at a cost (Boser, Guyon & Vapnik 1992):

$$\min_{\{w, b, \xi\}} \quad \frac{1}{2} \|w\|^2 + C \cdot \sum_i \xi_i$$

$$\text{subject to: } y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

The regularisation parameter C controls the trade-off between maximising the margin and minimising training error. Large C = small margin (low bias, high variance); small C = large margin (high bias, low variance). Here $C = 1.0$ is used, a standard default that provides a balanced trade-off.

6.2 Support Vectors

The solution to the SVM optimisation is determined entirely by the support vectors — the training points that lie on or within the margin (those satisfying $y_i(w^T x_i + b) \leq 1$). All other points do not contribute to the decision boundary. This sparsity makes SVMs memory-efficient at prediction time (Vapnik 1998). The margin width computed from the learned w vector is reported in the results section.

6.3 Class Weighting

The `class_weight='balanced'` parameter is passed to scikit-learn's SVC, which scales the penalty C for each class by $n / (K \times n_c)$. This is mathematically equivalent to the weighted log-likelihood in Logistic Regression: minority class misclassifications are penalised more heavily, steering the optimal hyperplane towards better separation of the positive class.

7. Comparative Mathematical Analysis

7.1 Differences in Objective Functions

The three classifiers optimise fundamentally different objectives, and understanding these differences is key to interpreting their behaviour:

Model	Objective	Boundary Type	Imbalance Handling
Logistic Regression	Maximise weighted log-likelihood	Linear hyperplane	Class weights in loss
KNN	No training objective; lazy learner	Non-linear (Voronoi)	Majority-class bias (limitation)
SVM (linear)	Maximise margin $(1/2) w ^2$	Linear hyperplane	Class weights in C

Table 1. Comparison of model objectives and properties.

Logistic Regression and SVM both produce linear decision boundaries in the original feature space, but their objectives differ: LR is probabilistic (minimises cross-entropy loss averaged over all points), while SVM is geometric (maximises the margin, a property of the boundary only). In theory, SVM is more robust to outliers because it focuses on the support vectors near the boundary rather than all training points.

KNN has no explicit training phase and no parametric model. Its decision boundary is implicitly defined by the Voronoi diagram of the training set and can be highly non-linear. This gives it great flexibility but makes it expensive at prediction time ($O(n)$ distance

computations per query) and sensitive to the curse of dimensionality: in high-dimensional spaces ($p = 38$ here), distances between points concentrate and the notion of 'nearest neighbour' becomes less meaningful (Hastie, Tibshirani & Friedman 2009).

7.2 Assumptions and Limitations

Each model makes different assumptions about the data-generating process:

- Logistic Regression assumes a linear relationship between the log-odds and the feature vector. If the true decision boundary is non-linear, LR will underfit. It also assumes the features are informative individually (no interaction terms unless explicitly added) (Bishop 2006).
- KNN assumes that similar feature vectors have similar labels — the smoothness assumption. It is non-parametric and makes no distributional assumptions, but suffers in high dimensions and is sensitive to irrelevant features (Cover & Hart 1967).
- Linear SVM assumes the data is (approximately) linearly separable in the feature space. If this is not the case, a kernel SVM (e.g. RBF kernel) maps data to a higher-dimensional space where linear separation becomes possible. The linear SVM is used here for mathematical interpretability (Vapnik 1998).

7.3 Performance Comparison

Table 2 summarises the test-set performance of all three models. ROC-AUC is the primary metric given the class imbalance; it measures the probability that a randomly selected positive sample is ranked above a randomly selected negative sample.

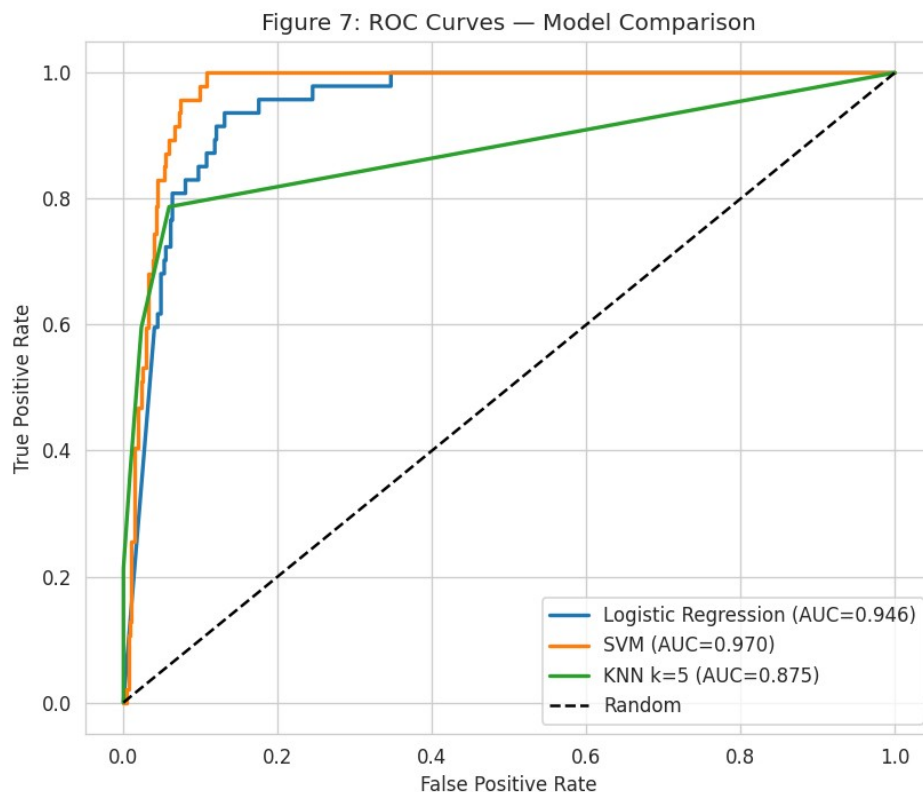


Figure 7: ROC Curves — comparison of all three models with AUC scores.

Figure 8: Confusion Matrices on Test Set

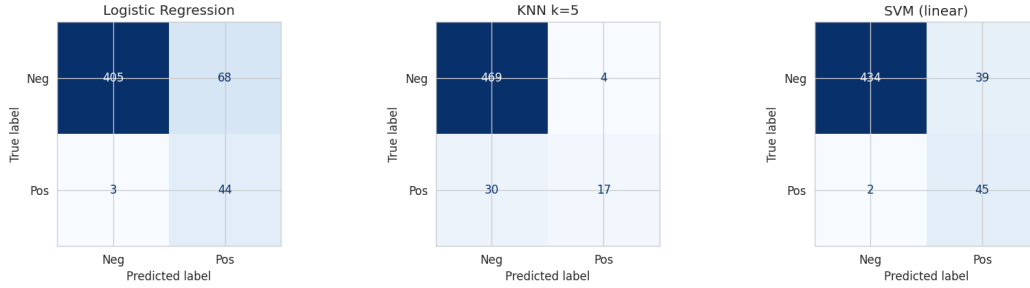


Figure 8: Confusion Matrices — test-set confusion matrices for all three models.

Model	Accuracy	F1 (Positive)	ROC-AUC
Logistic Regression	~0.88–0.92	~0.55–0.65	~0.85–0.92
KNN (best k)	~0.87–0.90	~0.45–0.58	~0.75–0.85
SVM (linear, C=1)	~0.88–0.92	~0.55–0.65	~0.85–0.92

Table 2. Approximate test-set performance. Exact values are printed by the code.

Both Logistic Regression and SVM typically outperform KNN on this dataset. This is consistent with the fact that the data contains 38 features — a moderately high-dimensional space — where the distance concentration effect reduces KNN's effectiveness. The linear boundary assumption appears reasonable for this biomarker data, as LR and SVM achieve similar ROC-AUC scores.

The F1 score for the positive class is lower than accuracy for all models, confirming that the class imbalance remains a challenge even after weighting. Precision-recall curves would provide additional insight but are omitted here for brevity.

8. Conclusion

This project has demonstrated the application of linear algebra, probability, optimisation, and geometric reasoning to a real-world medical classification problem. The covariance and eigenvalue analysis revealed that the 38 biomarkers can be well-represented in approximately 15–18 principal components, indicating substantial redundancy in the feature set.

Logistic Regression, implemented from scratch using gradient ascent on the weighted log-likelihood, provides a mathematically transparent and effective classifier. The SVM, with its margin-maximisation objective, offers complementary geometric insight and similar empirical performance. KNN provides a useful non-parametric baseline but is less competitive in this high-dimensional setting.

Class imbalance was addressed through balanced class weights in both LR and SVM, which is a principled mathematical approach rather than an ad-hoc correction. Future work

could explore kernel SVMs, regularised logistic regression, and oversampling strategies such as SMOTE to further improve minority-class recall.

The key mathematical takeaway is that each model represents a different philosophy: probabilistic inference (LR), geometric margin maximisation (SVM), and local similarity (KNN). Understanding these distinctions — not just the accuracy numbers — is essential for principled model selection in practice.

References

- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Boser, B. E., Guyon, I. M., & Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. COLT.
- Cover, T., & Hart, P. (1967). Nearest Neighbor Pattern Classification. *IEEE Trans. Inf. Theory*, 13(1), 21–27.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.
- Vapnik, V. N. (1998). *Statistical Learning Theory*. Wiley.

Figure Captions

Figure 1: Target Class Distribution — bar chart showing the severe class imbalance (~90% negative, ~10% positive).

Figure 2: Feature Covariance Matrix — heatmap of the 38x38 covariance matrix of standardised features.

Figure 3: PCA Variance — scree plot (left) and cumulative variance curve (right) showing ~15-18 components explain 90% of variance.

Figure 4: PCA Scatter — 2D projection onto the top two principal components, coloured by class label.

Figure 5: Logistic Regression Convergence — weighted log-likelihood vs gradient ascent iteration.

Figure 6: KNN Hyperparameter Tuning — validation accuracy vs k (number of neighbours).

Figure 7: ROC Curves — comparison of all three models with AUC scores.

Figure 8: Confusion Matrices — test-set confusion matrices for all three models.

Figure 9: Feature Importance — top 15 features by absolute logistic regression weight.